

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
Кафедра інформаційних технологій

ЛАБОРАТОРНА РОБОТА №1

З дисципліни: «Об'єктно-орієнтоване та подійно-орієнтоване програмування»
На тему: «Знайомство з базовим синтаксисом та операторами Java»

Роботу виконав:
студент II курсу групи АД-251
спеціальності F6
Горбунова А.А.

Роботу перевірів:
к т.н., доцент Рудніченко М.Д.

Оцінка: _____

1 МЕТА РОБОТИ ТА ЗАВДАННЯ

Мета роботи:

- 1) Встановити необхідне програмне забезпечення для програмування на ЯП Java;
- 2) Ознайомитися з базовим синтаксисом мови та основними операторами;
- 3) Вивчити приклади угоди щодо коду для ЯП Java.

Завдання на лабораторну роботу:

- 1) Напишіть метод, який приймає на вхід масив цілих чисел (довжиною 2 і більше) і повертає true, якщо в масиві кожен елемент більше або дорівнює попередньому. Інакше він повертає false.
- 2) Напишіть метод, який виводить на екран числа від 1 до 100. При цьому замість чисел, кратних трьох, програма повинна виводити слово Fizz, а замість чисел, кратних п'яти - слово Buzz. Якщо число кратне і 3, і 5, програма повинна виводити слово «FizzBuzz».
- 3) Реалізуйте метод, який приймає на вхід масив цілих чисел (довжиною 2 або більше) і повертає true, якщо масив можна розділити так, щоб сума чисел в обох частинах була рівною і false інакше.

2 ОБҐРУНТУВАННЯ ВИБОРУ ТА НАЛАШТУВАННЯ СЕРЕДОВИЩА РОЗРОБКИ

2.1 Аналіз інструментарію та обґрунтування вибору VS Code

Згідно з методичними вказівками, для розробки програмного забезпечення на мові Java традиційно розглядається “велика трійка” інтегрованих середовищ розробки (IDE): NetBeans, Eclipse та IntelliJ IDEA. Проте в межах виконання даної лабораторної роботи було прийнято рішення відмовитися від стандартних автоматизованих майстрів генерації проектів на користь легковажного та універсального текстового редактора **Visual Studio Code (VS Code)**.

Такий підхід має низку суттєвих інженерних та освітніх переваг:

- **Глибоке розуміння процесів компіляції:** Використання автоматизованих систем збирання (Maven, Gradle) на початковому етапі приховує від розробника внутрішні механізми взаємодії компілятора та віртуальної машини. Створення проекту “з нуля” дозволяє повністю контролювати структуру каталогів та процес збирання на рівні утиліт CLI.
- **Єдиний робочий простір (All-in-One):** VS Code дозволяє об’єднати розробку вихідного коду мовою Java та верстку технічного звіту (протоколу) в системі комп’ютерного набору LaTeX в межах одного вікна редактора.
- **Ефективність використання ресурсів:** На відміну від важких IDE, VS Code має високу швидкість роботи та низькі вимоги до оперативної пам’яті, забезпечуючи при цьому повноцінну підтримку мовного сервера Java.

2.2 Встановлення програмного забезпечення та конфігурація системи

Процес розгортання робочого оточення складався з наступних етапів:

- 1) **Встановлення інструментарію Java:** У систему було встановлено актуальний комплект розробника

Java Development Kit версії 27 (JDK-27), який включає компілятор `javac` та виконавче середовище JVM.

2) **Розширення VS Code:** Для забезпечення інтелектуального аналізу коду (IntelliSense) та підсвічування синтаксису в редактор було інтегровано офіційний пакет розширень *Extension Pack for Java* від Microsoft.

3) **Налаштування локалізації терміналу:** Для коректного відображення унікальних літер українського алфавіту *і, ї, є* у консолі Windows, сесію оболонки було програмно переведено на роботу зі стандартом кодування UTF-8.

2.3 Автоматизація збирання та формування єдиного циклу розробки

Для повної автоматизації рутинних операцій компіляції коду та генерації звіту було створено кастомний сценарій планувальника завдань всередині файлу `.vscode/tasks.json`. Облочкою виконання було обрано середовище Windows PowerShell.

Розроблена конфігурація об'єднує два послідовних процеси (Завдання) в один ультимативний цикл збирання:

1) **Завдання компіляції та запуску Java:** Компілює всі вихідні файли пакету за допомогою `javac` із примусовим прапорцем `-encoding UTF-8`, зберігає бінарні файли `.class` у каталог `bin`, ініціалізує UTF-8 кодування консолі та запускає віртуальну машину `java.exe` для виконання класу `Main`.

2) **Завдання верстки звіту:** Переходить у каталог `report` та викликає компілятор `xelatex` із прапорцем `-shell-escape` для генерації підсумкового PDF-документа, автоматично підтягуючи оновлені лістинги коду.

Завдяки такій конфігурації, натисканням однієї комбінації клавіш `Ctrl + Shift + B` виконується повний цикл: від перевірки працездатності алгоритмів Java до формування фінального друкованого протоколу лабораторної роботи.

2.4 Реалізація концепції базового пакету та структури проєкту вручну

Згідно з методичними вказівками, під час створення проєкту в IntelliJ IDEA особлива увага приділяється налаштуванню параметра **Base package** (базового пакету) у форматі `[назва_групи] . [прізвище]` (наприклад, `ad251.gorbunova`) **1**. У мові Java концепція пакетів (`package`) виконує роль просторів імен і логічного групування класів, але її фундаментальною особливістю є те, що **структура пакетів в коді повинна строго відповідати фізичній структурі папок на диску**.

Під час кастомного налаштування проєкту у VS Code цей процес не приходився за автоматичними майстрами IDE, а був реалізований повністю вручну за такими етапами:

1) Створення фізичної ієрархії каталогів: У корені робочого простору було створено каталог вихідного коду `src`, всередині якого вручну розгорнуто ланцюжок папок, що точно відповідає структурі базового пакету: `src/ad251/gorbunova/`.

2) Конфігурація логічного кореня (Source Root): Для того щоб мовний сервер Java у VS Code коректно інтерпретував ієрархію, у файлі налаштувань середовища `.vscode/settings.json` було явно вказано шлях до вихідних файлів за допомогою параметра:

```
"java.project.sourcePaths": ["src"]
```

Завдяки цьому середовище починає відлік простору імен саме з папки `src`, і шлях `src/ad251/gorbunova/Main.java` стає повністю еквівалентним оголошенню `package ad251.gorbunova;` в першому рядку кожного файлу.

3) Ізоляція артефактів збирання: На відміну від стандартного підходу, де скомпільовані файли суміщаються з вихідним кодом, за допомогою прапорця `-d bin` утиліти `javac` було налаштовано автоматичне дзеркальне створення структури пакетів у відокремленому бінарному каталозі `bin/ad251/gorbunova/`.

На рисунку 2.1 представлено фінальну архітектуру та дерево папок розробленого проєкту, згенероване безпосередньо в робочому просторі.

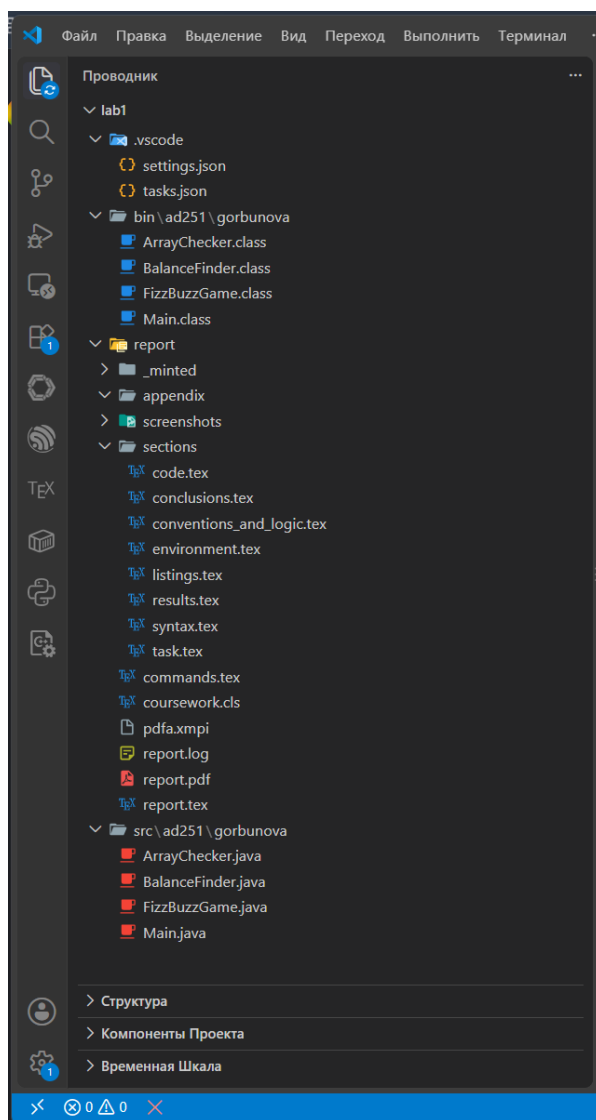


Рисунок 2.1 – Фізична структура каталогів проєкту із реалізацією базового пакету `ad251.gorbunova`

3 АНАЛІЗ БАЗОВОГО СИНТАКСИСУ ТА ОПЕРАТОРІВ: ПОРІВНЯЛЬНИЙ АСПЕКТ ІЗ C/C++

3.1 Загальні синтаксичні конструкції та типи даних

Оскільки синтаксис мови Java історично базується на мовах C та C++, базові керуючі конструкції (цикли `for`, `while`, `do-while` та оператори розгалуження `if-else`, `switch`) є практично ідентичними. Проте на рівні базової архітектури та типізації мови мають суттєві відмінності:

- **Строга типізація та незалежність від платформи:** На відміну від C/C++, де розмір типу `int` чи `long` залежить від архітектури процесора та компілятора, у Java всі примітивні типи мають фіксований розмір у пам'яті на будь-якій платформі (наприклад, `int` завжди 32 біти, `long` — 64 біти).
- **Відсутність беззнакових типів:** У Java відсутній модифікатор `unsigned`. Усі числові примітивні типи (включаючи `byte` та `short`) є виключно знаковими.
- **Булевий тип даних:** На відміну від C, де логічна умова може оперувати цілими числами (де 0 — це `false`, а будь-яке інше число — `true`), у Java тип `boolean` є ізольованим. Конструкції розгалуження приймають виключно значення `true` або `false`, що унеможлиблює помилки вигляду `if (a = b)`.

3.2 Ключові відмінності в роботі з пам'яттю та об'єктами

В межах виконання лабораторної роботи було досліджено три фундаментальні відмінності Java від мов C/C++, які критично впливають на проектування програм:

1) **Повна відсутність вказівників:** У Java повністю вилучено арифметику вказівників, прямий доступ до адрес пам'яті та оператори `*`, `&`, `->`. Усі об'єкти та масиви є посилальними типами й керуються через безпечні посилання.

2) **Автоматичне керування пам'яттю (Garbage Collection):** Розробнику більше не потрібно вручну звільняти пам'ять за допомогою `free()` чи `delete`. Об'єкти

створюються у кучі (heap) за допомогою ключового слова `new`, а збірник сміття (GC) автоматично утилізує їх, коли на них не залишається активних посилань.

3) **Масиви як повноцінні об'єкти:** На відміну від C/C++, де масив є просто вказівником на перший елемент і вимагає окремої передачі його розміру у функції, в Java масив є об'єктом. Він інкапсулює у собі властивість `length`, що дозволяє безпечно контролювати межі ітерацій і унеможлиблює помилку переповнення буфера (`ArrayIndexOutOfBoundsException`).

4 СТАНДАРТИ ОФОРМЛЕННЯ КОДУ ТА АРХІТЕКТУРА АЛГОРИТМІВ

4.1 Дотримання конвенції Oracle Code Conventions

У межах виконання лабораторної роботи особливу увагу було приділено стандартам інженерного оформлення вихідного коду згідно з вимогами *Oracle Code Conventions*. Було опановано та застосовано на практиці такі правила:

- **Угода про іменування (Naming Conventions):** Імена всіх створених компонентів та класів (`ArrayChecker`, `FizzBuzzGame`, `BalanceFinder`) записані у стилі `UpperCamelCase`. Назви методів та змінних (`isSortedUp()`, `totalSum`, `leftSum`) строго відповідають стилю `lowerCamelCase`, де перше слово обов'язково є дієсловом для методів та іменником для змінних.

- **Стандартизація коментарів:** Структура кожного файлу містить три рівні документування:

- 1) *Блок Copyright* у самому верху файлу із зазначенням ліцензії Apache 2.0 та автора.

- 2) *JavaDoc коментарі* (`/** ... */`) перед класами та методами із застосуванням тегів `@author`, `@version`, `@param` та `@return`.

- 3) *Внутрішні коментарі* (`/* ... */` та `//`) для пояснення складних ділянок бізнес-логіки з обов'язковим відступом у один порожній рядок перед ними.

5 СТРУКТУРА ПРОЕКТУ, ЛОГІКА ЗАВДАНЬ ТА ВИХІДНИЙ КОД ПРОГРАМИ

На відміну від процедурного підходу, кожне завдання лабораторної роботи було інкапсульовано в окремий автономний клас-компонент, що дозволило уникнути використання статичних методів у стилі мови C.

1) Перевірка впорядкованості масиву (Клас **ArrayChecker**):

Алгоритм виконує один лінійний прохід по масиву починаючи з індексу 1. Якщо поточний елемент виявляється меншим за попередній, метод негайно повертає `false`, що оптимізує швидкість виконання.

2) Матричне виведення FizzBuzz (Клас **FizzBuzzGame**):

Для створення компактного та зручного для звітності інтерфейсу, логіку перевірки відповідно до заданої кратності чисел (на 3, 5 та їхній добуток) було об'єднано з формативним виведенням `System.out.printf("%-10s")`. Це дозволило згенерувати результати у вигляді математичної сітки розміром 10 на 10 стовпців.

3) Пошук балансу сум (Клас **BalanceFinder**):

Спочатку обчислюється повна сума масиву, а під час другого лінійного проходу накопичується ліва частина. Рівновагу знайдено в момент, коли ліва сума стає рівною рівно половині загальної суми.

4) Координація та точка входу (Клас **Main**):

Клас містить обов'язковий для виконання Java-додатків метод `public static void main(String[] args)`. Назначенням цього компонента є повне управління життєвим циклом програми ініціалізує тестові масиви даних, виконує обчислення та створює екземпляри класів `ArrayChecker`, `FizzBuzzGame` та `BalanceFinder`, передає їм аргументи через конструктори та виводить фінальні результати обчислень у термінал середовища розробки.

6 ЛІСТИНГ ПРОГРАМНОГО КОДУ

У цьому розділі наведено повний вихідний код розробленої структури класів. Код оформлено згідно зі стандартами *Oracle Code Conventions* та містить вичерпну документацію у форматі *JavaDoc*.

6.1 Клас Main.java

```

1  /*
2  * Copyright (c) 2026 Anastasiia Gorbunova
3  * National University "Odesa Polytechnic"
4  *
5  * Licensed under the Apache License, Version 2.0 (the "License");
6  * you may not use this file except in compliance with the License.
7  * You may obtain a copy of the License at
8  *
9  *     http://apache.org
10 *
11 * Unless required by applicable law or agreed to in writing, software
12 * distributed under the License is distributed on an "AS IS" BASIS,
13 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
14 * See the License for the specific language governing permissions and
15 * limitations under the License.
16 */
17 package ad251.gorbunova;
18
19 /**
20 * Головний керуючий клас програми (Точка входу).
21 * Координує створення об'єктів-компонентів та демонстрацію виконання завдань 1, 2 та 3*.
22 *
23 * @author Анастасія Горбунова
24 * @version 1.1 05 Oct 2026
25 */
26 public class Main {
27
28     /**
29     * Точка входу в програму. Викликає методи компонентів з демонстраційними даними.
30     *
31     * @param args аргументи командного рядка при запуску додатка
32     */
33     public static void main(String[] args) {
34         // Перед коментарем прийнято залишати порожній рядок
35         // --- Демонстрація та перевірка Завдання 1 ---
36         System.out.print("=== Лабораторна робота №1 ===\n\n");
37         System.out.print("Завдання 1 (Перевірка масиву на впорядкованість):\n");
38
39         int[] sortedData = {1, 2, 3, 3, 5};
40         int[] unsortedData = {1, 3, 2, 4, 5};
41
42         ArrayChecker correctChecker = new ArrayChecker(sortedData);
43         ArrayChecker wrongChecker = new ArrayChecker(unsortedData);
44
45         System.out.print("    Масив {1, 2, 3, 3, 5} -> " + correctChecker.isSortedUp() + "\n");
46         System.out.print("    Масив {1, 3, 2, 4, 5} -> " + wrongChecker.isSortedUp() + "\n\n");
47
48         // --- Демонстрація та перевірка Завдання 2 ---
49         System.out.print("Завдання 2 (Сітка FizzBuzz для чисел від 1 до 100):\n");
50
51         FizzBuzzGame game = new FizzBuzzGame(1, 100);
52         game.play();
53         System.out.print("\n");
54
55         // --- Демонстрація та перевірка Завдання 3* ---
56         System.out.print("Завдання 3* (Пошук точки балансу сум масиву):\n");
57
58         int[] balanceData1 = {1, 1, 1, 2, 1};
59         int[] balanceData2 = {2, 1, 1, 2, 1};
60         int[] balanceData3 = {10, 10};

```

```
61
62     BalanceFinder finder1 = new BalanceFinder(balanceData1);
63     BalanceFinder finder2 = new BalanceFinder(balanceData2);
64     BalanceFinder finder3 = new BalanceFinder(balanceData3);
65
66     System.out.print("    Method({1, 1, 1, 2, 1}) -> " + finder1.canBalance() + "\n");
67     System.out.print("    Method({2, 1, 1, 2, 1}) -> " + finder2.canBalance() + "\n");
68     System.out.print("    Method({10, 10})      -> " + finder3.canBalance() + "\n");
69 }
70 }
```

6.2 Клас ArrayChecker.java

```

1  /*
2  * Copyright (c) 2026 Anastasiia Gorbunova
3  * National University "Odesa Polytechnic"
4  *
5  * Licensed under the Apache License, Version 2.0 (the "License");
6  * you may not use this file except in compliance with the License.
7  * You may obtain a copy of the License at
8  *
9  *     http://apache.org
10 *
11 * Unless required by applicable law or agreed to in writing, software
12 * distributed under the License is distributed on an "AS IS" BASIS,
13 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
14 * See the License for the specific language governing permissions and
15 * limitations under the License.
16 */
17 package ad251.gorbunova;
18
19 /**
20 * Клас-компонент для інспекції та аналізу властивостей масивів.
21 * Використовується для перевірки математичного порядку елементів.
22 *
23 * @author Анастасія Горбунова
24 * @version 1.1 05 Oct 2026
25 */
26 public class ArrayChecker {
27     /** Внутрішній масив цілих чисел для дослідження */
28     private final int[] array;
29
30     /**
31      * Конструктор для ініціалізації об'єкта аналізованим масивом.
32      *
33      * @param array масив цілих чисел (за умовою довжиною від 2 елементів)
34      */
35     public ArrayChecker(int[] array) {
36         this.array = array;
37     }
38
39     /**
40      * Перевіряє, чи є кожен елемент масиву більше або дорівнює попередньому.
41      * Реалізує алгоритм перевірки масиву на впорядкованість за неубуванням.
42      *
43      * @return true, якщо масив відсортований за зростанням/неубуванням, інакше false
44      */
45     public boolean isSortedUp() {
46         // Перед коментарем прийнято залишати порожній рядок
47         // Валідація вхідних даних на випадок порожнього посилання або короткого масиву
48         if (array == null || array.length < 2) {
49             return false;
50         }
51
52         /**
53          * Цикл починається з другого елемента (індекс 1).
54          * Порівнюємо поточний елемент із безпосередньо попереднім.
55          */
56         for (int i = 1; i < array.length; i++) {
57             if (array[i] < array[i - 1]) {
58                 return false; /* Знайдено порушення порядку сортування */
59             }
60         }
61
62         return true;
63     }
64 }

```

6.3 Клас FizzBuzzGame.java

```

1  /*
2  * Copyright (c) 2026 Anastasiia Gorbunova
3  * National University "Odesa Polytechnic"
4  *
5  * Licensed under the Apache License, Version 2.0 (the "License");
6  * you may not use this file except in compliance with the License.
7  * You may obtain a copy of the License at
8  *
9  *     http://apache.org
10 *
11 * Unless required by applicable law or agreed to in writing, software
12 * distributed under the License is distributed on an "AS IS" BASIS,
13 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
14 * See the License for the specific language governing permissions and
15 * limitations under the License.
16 */
17 package ad251.gorbunova;
18
19 /**
20 * Клас, що реалізує логіку відомої у Британії дитячої гри FizzBuzz.
21 * Використовує формативне виведення для створення акуратної текстової сітки.
22 *
23 * @author Анастасія Горбунова
24 * @version 1.1 05 Oct 2026
25 */
26 public class FizzBuzzGame {
27     /** Початкове значення діапазону чисел гри */
28     private final int start;
29     /** Кінцеве значення діапазону чисел гри */
30     private final int end;
31
32     /**
33      * Конструктор для ініціалізації меж ігрового діапазону.
34      *
35      * @param start перше число діапазону
36      * @param end останнє число діапазону
37      */
38     public FizzBuzzGame(int start, int end) {
39         this.start = start;
40         this.end = end;
41     }
42
43     /**
44      * Запускає гру та виводить результат у вигляді красивої сітки (10 стовпців).
45      * Замінює числа кратні 3 на Fizz, кратні 5 на Buzz, кратні обом – на FizzBuzz.
46      */
47     public void play() {
48         int count = 0; /* Лічильник виведених елементів у поточному рядку */
49
50         for (int i = start; i <= end; i++) {
51             String value; /* Текстове представлення поточного кроку гри */
52
53             /*
54              * Блок аналізу кратності числа за допомогою оператора залишку від ділення.
55              * Спочатку перевіряємо комбіновану умову кратності обом числам (3 та 5).
56              */
57             if (i % 3 == 0 && i % 5 == 0) {
58                 value = "FizzBuzz";
59             } else if (i % 3 == 0) {
60                 value = "Fizz";
61             } else if (i % 5 == 0) {
62                 value = "Buzz";
63             } else {
64                 value = String.valueOf(i);
65             }
66
67             // Перед однорядковим коментарем прийнято залишати порожній рядок
68             // %-10s вирівнює текст по лівому краю, резервуючи під нього 10 символів
69             System.out.printf("%-10s", value);
70             count++;
71
72             // Перевіряємо, чи заповнився поточний рядок сітки з 10 елементів
73             if (count % 10 == 0) {
74                 System.out.print("\n");
75             }
76         }
77     }

```

```
78         // Перевіряємо необхідність фінального перенесення рядка в самому кінці
79         if (count % 10 != 0) {
80             System.out.print("\n");
81         }
82     }
83 }
```

6.4 Клас BalanceFinder.java

```

1  /*
2  * Copyright (c) 2026 Anastasiia Gorbunova
3  * National University "Odesa Polytechnic"
4  *
5  * Licensed under the Apache License, Version 2.0 (the "License");
6  * you may not use this file except in compliance with the License.
7  * You may obtain a copy of the License at
8  *
9  *     http://apache.org
10 *
11 * Unless required by applicable law or agreed to in writing, software
12 * distributed under the License is distributed on an "AS IS" BASIS,
13 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
14 * See the License for the specific language governing permissions and
15 * limitations under the License.
16 */
17 package ad251.gorbunova;
18
19 /**
20 * Клас для обчислення та пошуку точок балансу суми елементів всередині масивів.
21 * Містить оптимізований алгоритм лінійного пошуку розділювача.
22 *
23 * @author Анастасія Горбунова
24 * @version 1.1 05 Oct 2026
25 */
26 public class BalanceFinder {
27     /** Масив цілих чисел для обчислення балансу частин */
28     private final int[] array;
29
30     /**
31      * Конструктор для прив'язки досліджуваного масиву до об'єкта обчислювача.
32      *
33      * @param array масив цілих чисел довжиною 2 або більше
34      */
35     public BalanceFinder(int[] array) {
36         this.array = array;
37     }
38
39     /**
40      * Перевіряє можливість розділення масиву на дві частини з рівною сумою чисел.
41      * Оптимізовано під складність O(N) без використання вкладених циклів.
42      *
43      * @return true, якщо масив можна збалансувати, інакше false
44      */
45     public boolean canBalance() {
46         // Перед коментарем прийнято залишати порожній рядок
47         if (array == null || array.length < 2) {
48             return false;
49         }
50
51         int totalSum = 0;           /* Повна сума абсолютно всіх елементів масиву */
52         int leftSum = 0;           /* Накопичувана сума лівої частини при ітерації */
53
54         // Обчислюємо загальну суму масиву за допомогою компактного циклу foreach
55         for (int element : array) {
56             totalSum += element;
57         }
58
59         /*
60          * Якщо загальна сума чисел непарна, її математично неможливо
61          * розділити на дві рівні цілі частини. Відразу повертаємо false.
62          */
63         if (totalSum % 2 != 0) {
64             return false;
65         }
66
67         // Послідовно накопичуємо ліву суму та порівнюємо її з половиною від загальної
68         for (int element : array) {
69             leftSum += element;
70
71             if (leftSum == totalSum / 2) {
72                 return true; /* Точку рівноваги (балансу) знайдено */
73             }
74         }
75
76         return false;
77     }

```


7 РЕЗУЛЬТАТИ РОБОТИ ПРОГРАМИ

Для верифікації правильності роботи розроблених алгоритмів було здійснено запуск програми в єдиному циклі збирання середовища VS Code. На рисунку 7.1 представлено фінальне виведення результатів у термінал Windows PowerShell.

```

=== Лабораторна робота №1 ===

Завдання 1 (Перевірка масиву на впорядкованість):
Масив {1, 2, 3, 3, 5} -> true
Масив {1, 3, 2, 4, 5} -> false

Завдання 2 (Гра FizzBuzz для чисел від 1 до 100):
1      2      Fizz   4      Buzz   Fizz   7      8      Fizz   Buzz
11     Fizz   13     14     FizzBuzz 16     17     Fizz   19     Buzz
Fizz   22     23     Fizz   Buzz   26     Fizz   28     29     FizzBuzz
31     32     Fizz   34     Buzz   Fizz   37     38     Fizz   Buzz
41     Fizz   43     44     FizzBuzz 46     47     Fizz   49     Buzz
Fizz   52     53     Fizz   Buzz   56     Fizz   58     59     FizzBuzz
61     62     Fizz   64     Buzz   Fizz   67     68     Fizz   Buzz
71     Fizz   73     74     FizzBuzz 76     77     Fizz   79     Buzz
Fizz   82     83     Fizz   Buzz   86     Fizz   88     89     FizzBuzz
91     92     Fizz   94     Buzz   Fizz   97     98     Fizz   Buzz

Завдання 3* (Пошук точки балансу сум масиву):
Method({1, 1, 1, 2, 1}) -> true
Method({2, 1, 1, 2, 1}) -> false
Method({10, 10})       -> true
  
```

Рисунок 7.1 – Вигляд терміналу після успішного виконання всіх трьох завдань лабораторної роботи

Як видно з результатів тестування:

- Компонент `ArrayChecker` коректно визначив впорядкованість першого масиву (`true`) та зафіксував порушення порядку у другому (`false`).
- Гра `FizzBuzzGame` згенерувала компактну текстову матрицю розміром 10×10 елементів із правильним заміщенням чисел на відповідні мовні маркери.
- Оптимізований обчислювач `BalanceFinder` безпомилково знайшов точки рівноваги сум для демонстраційних масивів.

8 ВИСНОВКИ:

В ході виконання лабораторної роботи було успішно досягнуто поставлену мету: розгорнуто кастомне робоче оточення для програмування на мові Java та опановано базовий синтаксис і оператори цієї мови через призму порівняльного аналізу з екосистемою C/C++.

На основі виконаних досліджень та розробленого програмного комплексу сформульовано такі науково-практичні висновки:

1) **Ефективність інструментарію:** Ручне налаштування універсального редактора VS Code та інтеграція компілятора JDK-27 за допомогою сценаріїв автоматизації Windows PowerShell (`tasks.json`) дозволили створити прозорий та ефективний цикл збирання проєкту «з нуля». Такий підхід забезпечив повний контроль над структурою пакетів та процесом компіляції без використання ваговитих сторонніх систем збирання.

2) **Концептуальні переваги Java:** Практична реалізація алгоритмів підтвердила ключові архітектурні відмінності Java від мов C/C++. Відсутність ручного керування пам'яттю та вказівників (завдяки роботі Garbage Collector), строга ізоляція булевого типу та робота з масивами як з повноцінними об'єктами (властивість `length`) суттєво підвищують безпеку розробки та нівелюють ризики виникнення критичних помилок переповнення буфера.

3) **Об'єктно-орієнтована декомпозиція:** На відміну від процедурного стилю, завдання лабораторної роботи були успішно розділені на ізольовані класи-компоненти (`ArrayChecker`, `FizzBuzzGame`, `BalanceFinder`) із чіткою інкапсуляцією логіки, що заклало правильний фундамент для подальшого вивчення об'єктно-орієнтованого програмування.

4) **Культура розробки:** Весь вихідний код програми був повністю адаптований до суворих вимог міжнародного стандарту *Oracle Code Conventions*. Впровадження трирівневої системи коментування (ліцензійний блок, JavaDoc-документація та внутрішні пояснення) разом із автоматичною збіркою звіту в системі LaTeX дозво-

лили вивести фінальну технічну документацію на рівень промислових стандартів розробки.

9 ІНФОРМАЦІЙНІ РЕСУРСИ ТА ПОСИЛАННЯ НА ВИХІДНИЙ КОД

Згідно з вимогами до здачі лабораторної роботи, повний проєкт вихідного коду розробленого додатку розміщено у віддаленому репозиторії системи контролю версій.

– **Посилання на Git-репозиторій проєкту:**

<https://github.com/nasuehfw/oop-lab1.git>

– **Електронний каталог спільного доступу:**

[https://drive.google.com/drive/folders/1mIulRQSfoHAyQ2GnqdQbwGz2dFX-5VGK?
usp=drive_link](https://drive.google.com/drive/folders/1mIulRQSfoHAyQ2GnqdQbwGz2dFX-5VGK?usp=drive_link)